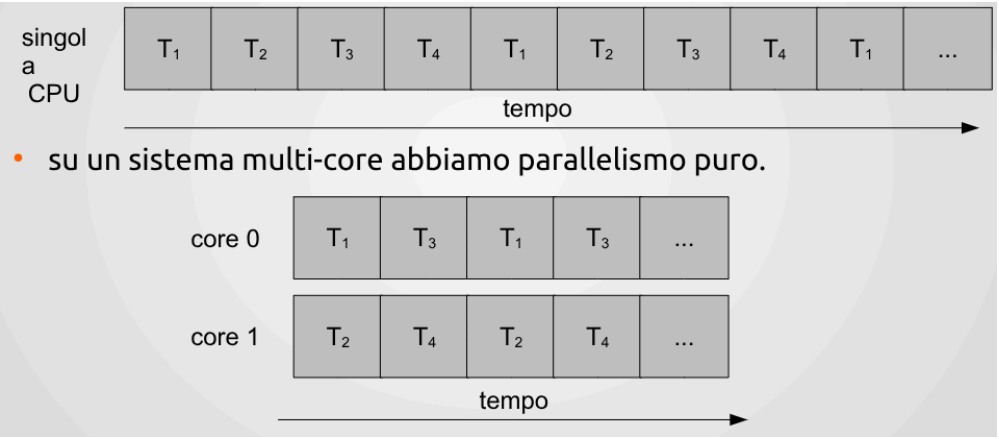
5_19_marzo

Thread

(continuo di 4_17_marzo)

Riassumendo le CPU di oggi hanno più core fisici, dove vengono eseguiti più processi/thread all'interno di ogni singolo core (contest switch) quindi ad oggi si sfrutta il parallelismo puro (tra core) e lo pseudo-parallelismo (all'interno di ogni singolo core).



Programmazione multicore

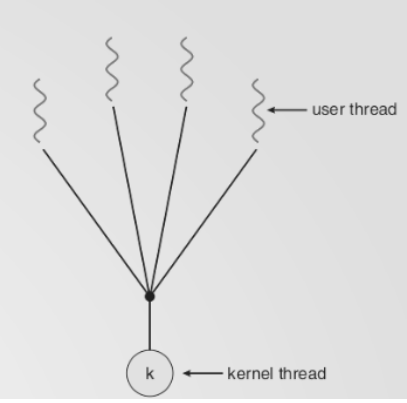
Programmare pensando ad un sistema multi thread, ci permette di non cambiare nulla quando arriviamo in uno scenario multicore, nella pratica non cambia nulla, i thread vengono divisi sui vari core.

Problemi della programmazione multicore
Scrivere programmi multi-core non è banale, dobbiamo stare attenti alle seguenti cose:

- **Separazione dei task**: per task intendiamo i compiti che la nostra CPU deve eseguire, è importante che questi vengano individuati e resi parallelizzabili, a grandi linee abbiamo un task per thread. Ha senso creare un thread per task pesanti.
- **Bilanciamento**: cioè la giusta suddivisione del lavoro tra i thread e i core
- **Suddivisione dei dati**: ognuno di questi task avrà bisogno di strutture dati che possono essere anche comuni (può capitare anche che più thread lavorano sulla stessa struttura dati)
- **Dipendenza dei dati**: visto che i dati possono essere condivisi tra i vari thread ci possono essere delle dipendenze tra un task (e quindi un thread) ed un altro, è importante tenere a mente questo dettaglio per cercare di creare thread indipendenti quando possibile.
- **Test e debugging**: Ci possono essere diversi problemi quando si parla di processi multicore. Dato lo stesso input si possono avere output diversi questo perché ci

possono essere problemi dovuti a come i thread vengono inseriti nello scheduler, questo ovviamente non va bene, il debugging e i test servono a cercare questo tipo di problemi.

Thread a livello utente



Le linee a zig-zag sono i thread generati dal programma in user mode (es. Excel) che confluiscono in un unico punto verso il kernel, *questo sta a indicare che il kernel non ha la minima idea di cosa sono questi thread lui li considera un unico processo*, quindi il programma usa una libreria che crea e gestisce una sua personale tabella dei thread (come se fosse un mini sistema operativo integrato)
PRO:

- scheduling personalizzato: il programma può scegliere le priorità tra i thread senza dover sottostare allo scheduler del sistema operativo
- il dispatching non richiede trap nel kernel: il cambio di thread è velocissimo perché il programma fa tutto nella user mode, senza dover passare alla kernel mode
  CONTRO:
- chiamate bloccanti:
  - *Chiamate di sistema bloccanti:* Quando un thread richiede un'operazione di I/O, l'operazione viene passata al kernel (trap). Dato che il sistema operativo vede e gestisce solo il processo nel suo insieme, mette in stato di attesa (*blocked*) l'intero processo finché l'operazione di I/O non è completata. Di conseguenza, tutti gli altri thread a livello utente si bloccano inevitabilmente, anche se avrebbero istruzioni pronte da eseguire e non dipendono da quel dato.
  - *Page fault:* Se un thread tenta di accedere a una variabile o a un'istruzione che si trova in un'area di memoria non attualmente caricata nella memoria RAM (ad esempio perché è stata spostata nell'area di swap su disco), si genera un'eccezione hardware chiamata *page fault*. Il sistema operativo deve intervenire per leggere la pagina mancante dal disco. Essendo un'operazione molto lenta, il kernel sospende l'esecuzione dell'intero processo. Pertanto, anche se gli altri thread del processo avessero tutte le loro pagine già pronte in RAM e potessero continuare a lavorare, vengono bloccati tutti fino al completamento del caricamento della pagina richiesta.

- nessun vero parallelismo tra i processi
- non si ha possibilità di prelazione, ovvero c'è la possibilità che la CPU o in generale una risorsa non venga rilasciata

Per gestire i thread a questo livello devo praticamente fare sempre le stesse cose, ma lo devo fare in maniera manuale a livello utente. Più precisamente il programmatore va a scrivere tutto il ciclo vitale di un thread e quando si può usando thread_yield il thread rilascia l'utilizzo della CPU ad un altro thread (dello stesso processo)

Questo modello viene chiamato modello "1 a molti"

Thread a livello kernel

Il kernel gestisce i thread, conosce che cosa sono i thread e sa che dentro ogni processo ci possono essere più thread, ovviamente il kernel deve essere affidabile nelle gestione dei thread, le chiamate di sistema sono analoghe a quelle usate a livello utente. Si ha un unica tabella dei thread a livello kernel.
PRO:

- Un thread con chiamata bloccante non va ad intralciare gli altri, come succede a livello user, proprio perché il kernel li può vedere.
- La prelazione è disponibile, quindi in caso di thread a livello kernel che occupano all'infinito una risorsa possono essere killati
  CONTRO:
- si ha un cambio di contesto più lento, perché il kernel va a gestire tutti i thread e non solo quelli di uno specifico processo (richiede trap)
- creazione e distruzione più costose (il numero di thread kernel è tipicamente limitato, è importante riciclarli)

Il riciclo dei thread ci evita di utilizzare le chiamate thread_create e thread_delete in questo modo si risparmia molto tempo.

Con questo tipo di gestione il nostro sistema operativo vede tutto come thread anche un processo che thread non ne ha.

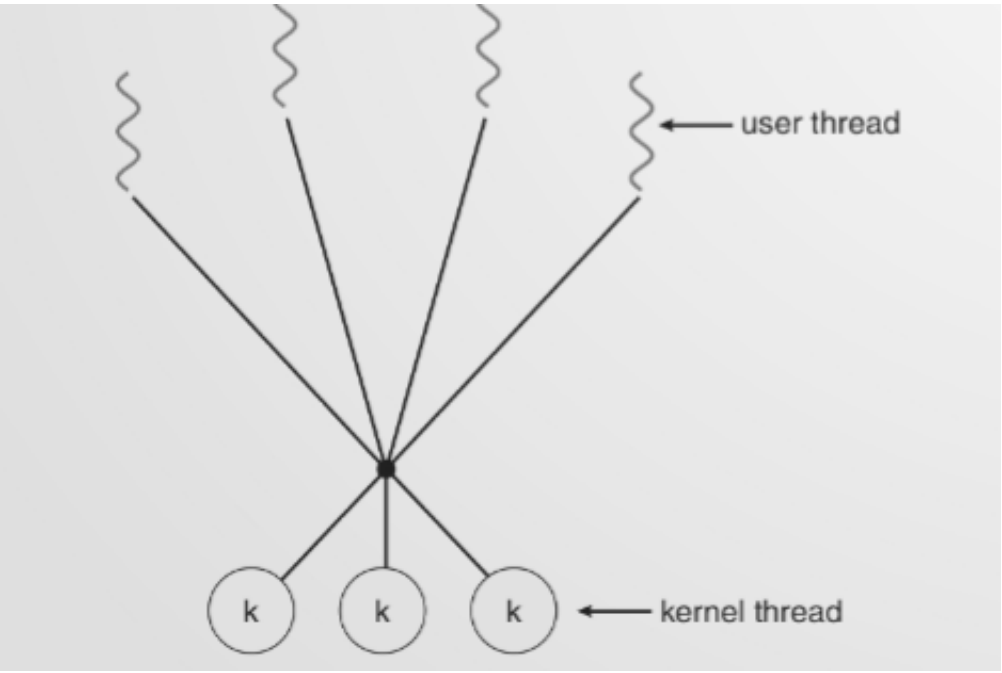
Questo modello viene chiamato modello "1 a 1"

Modello ibrido

Questo modello mira a combinare i punti di forza dell'approccio a livello utente (efficienza e flessibilità) con quelli dell'approccio a livello kernel (vero parallelismo e tolleranza ai blocchi), mitigando i difetti di entrambi.

In pratica, il sistema effettua un **multiplexing** di numerosi thread a livello utente su un numero limitato (uguale o inferiore) di thread a livello kernel.

- **Livello Utente:** Il programmatore, tramite la libreria, può creare tutti i thread necessari per la logica dell'applicazione. Il *context switch* tra questi thread rimane rapido perché avviene nello spazio utente.
- **Livello Kernel:** Il sistema operativo fornisce e gestisce un pool di thread a livello kernel, assegnandoli ai processori fisici disponibili. La libreria di runtime si occupa poi di mappare dinamicamente i thread utente sui thread kernel liberi.
  Il grande vantaggio: In questo modo, se un thread utente effettua una chiamata di sistema bloccante o subisce un *page fault*, il sistema operativo blocca solo il thread kernel associato in quel preciso momento. Gli altri thread a livello utente possono continuare la loro esecuzione appoggiandosi ai restanti thread a livello kernel disponibili.



Nei nostri sistemi operativi

Tutti i sistemi operativi moderni supportano thread a livello kernel, anche supporto a livello utente tramite diverse librerie:

- green threads su Solaris
- GNU portable thread su UNIX
- fiber su windows
  Esistono anche delle librerie di accesso ai thread, che ci aiutano ad interagire allo stesso modo con quest'ultimi indipendentemente dal sistema operativo usato

Comunicazione fra processi

Spesso i processi hanno bisogno di cooperare, abbiamo diversi modi per fare ciò:

- **Pipe**: collegamento tra processi, in pratica cmd1 | cmd2 | cmd3, quello che faccio è avviare tutti e tre i comandi in parallelo, quello che succede è l'output del primo va in input all'altro e cosi via

- **IPC (Inter Process Communication)**: dei processi con il proprio spazio di indirizzamento riescono a comunicare grazie a dei segmenti di memoria condivisa, questo permette la comunicazione tra processi in modo efficiente.

Possono verificarsi i seguenti problemi:

- come scambiarsi i dati
- accavallamento delle operazioni su dati comuni
- coordinamento tra le operazioni

Esempio: Supponiamo di avere due processi P1 e P2 che aggiornano una variabile intera condivisa *x*. Entrambi eseguono un ciclo `for` che incrementa *x* di uno (`x = x + 1`) per 100 iterazioni ciascuno.

L'istruzione `x = x + 1` non è atomica: il processore la scompone in tre passi distinti:

1. **Leggi** il valore di *x* dalla memoria
2. **Incrementa** il valore nel registro
3. **Scrivi** il nuovo valore in memoria

Il problema nasce quando i due processi si intersecano in mezzo a questi passi:

- *x* = 0
- P1 legge x → ottiene 0
- P2 legge x → ottiene 0 *(prima che P1 abbia scritto!)*
- P1 incrementa → 0+1 = 1, scrive x = 1
- P2 incrementa → 0+1 = 1, scrive x = 1

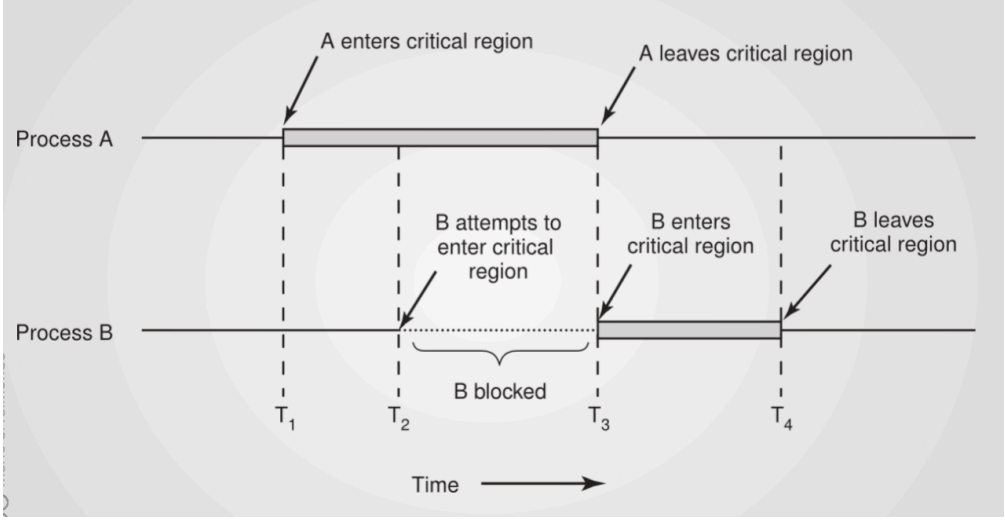
Risultato: *x* vale 1 invece di 2 — un incremento è andato perso. Questo può ripetersi in qualsiasi iterazione, quindi al termine delle 100 iterazioni di entrambi i processi, il valore finale di *x* potrebbe essere molto inferiore a 200. Questo fenomeno prende il nome di **race condition** (corsa critica). Un altro esempio è quello dell'accredito di denaro sul conto bancario.

Sezioni critiche: Per evitare le race condition, il programmatore deve garantire che i processi non accedano simultaneamente alla stessa risorsa condivisa. La porzione di codice in cui avviene questo accesso è detta *sezione critica*, e deve essere eseguita in mutua esclusione: un solo processo alla volta può trovarsi al suo interno.

Per avere una buona soluzione dobbiamo rispettare queste quattro condizioni:

1. mutua esclusione nell'accesso alle sezioni critiche
2. nessuna assunzione sulla velocità di esecuzione o sul numero di CPU
3. nessun processo fuori dalla propria sezione critica può bloccare un altro processo

4. nessun processo dovrebbe restare all'infinito in attesa di entrare nella propria sezione critica
   Di seguito il dettaglio di quello che succede realmente



Ci sono diverse soluzioni a questo problema, come:

- Disabilitare gli interrupt
- Variabili di lock
- Alternanza stretta: (o *strict alternation*) è un algoritmo software elementare utilizzato per garantire la **mutua esclusione** tra due processi concorrenti che devono accedere a una zona di memoria o a una risorsa condivisa, definita **sezione critica**. Il meccanismo si basa su un'unica variabile globale condivisa, tipicamente un intero chiamato `turno`, che indica esplicitamente quale processo ha il diritto di entrare nella propria sezione critica.

```
int N=2
int turn

function enter_region(int process)
    while (turn != process) do
        nothing

function leave_region(int process)
    turn = 1 - process
```